Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

“Розробка інтерфейсу користувача”

Виконав:

студент групи ІМ-о51
Тараєв Олексій Богданович
Номер в списку - 14

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання:

$$Ж = Ж_{лаб2} + 1 = 14 + 1 = 15$$

Масив покажчиків Shape (п. 4):

Гранична місткість: $N = Ж + 100 = 15 + 100 = 115$ **об'єктів.**

Тип масиву (15 bmod 3 = 0): динамічний масив (Shape **pcshape / динамічний список list[Shape]).

«Гумовий» слід при вводі (п. 5):

15 bmod 4 = 3 — пунктирна лінія чорного кольору.

Прямокутник (п. 6.1):

Введення (15 bmod 2 = 1): від центру до одного з кутів.

Відображення (15 \bmod 5 = 0): чорний контур з білим заповненням.

Еліпс (п. 6.2):

Введення (15 \bmod 2 = 1): по двох протилежних кутах охоплюючого прямокутника.

Відображення (15 \bmod 5 = 0): чорний контур еліпса без заповнення.

Позначка поточного типу об'єкта (п. 7):

15 \bmod 2 = 1 — у заголовку вікна.

Вимоги до інтерфейсу (Лаб. 3, п. 2–3):

Реалізація панелі інструментів Toolbar з кнопками-іконками, що візуально дублюють підпункти меню «Об'єкти».

Спливаючі підказки (tooltips) для кнопок панелі.

Обов'язкова інкапсуляція обробників подій, зокрема OnNotify, як функцій-членів відповідних класів.

код

editor.py

```
import logging
import tkinter as tk
from abc import ABC, abstractmethod
from typing import Any


from shape import EllipseShape, LineShape, PointShape, RectShape


__all__ = [
    "Editor",
    "EllipseEditor",
    "LineEditor",
    "PointEditor",
    "RectEditor",
    "ShapeEditor",
]


logger = logging.getLogger("Lab3.Editor")
```

```
class Editor(ABC):
    @abstractmethod
    def on_lb_down(self, event: tk.Event) -> None:
        pass

    @abstractmethod
    def on_lb_up(self, event: tk.Event) -> None:
        pass

    @abstractmethod
    def on_mouse_move(self, event: tk.Event) -> None:
        pass

    @abstractmethod
    def on_paint(self, canvas: tk.Canvas) -> None:
        pass



class ShapeEditor(Editor):
    def __init__(self, owner: Any) -> None:
        self._owner = owner
        self._x_start: int = 0
        self._y_start: int = 0
        self._x_end: int = 0
        self._y_end: int = 0
        self._is_drawing: bool = False
```

```
    def on_paint(self, canvas: tk.Canvas) -> None:
        pass

    def _clear_rubber_band(self) -> None:
        self._owner.canvas.delete("rubber_band")


class PointEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("PointEditor mouse down at (%d, %d).", event.x,
event.y)
        shape = PointShape()
        shape.set(event.x, event.y, event.x, event.y)
        self._owner.add_shape(shape)

    def on_lb_up(self, event: tk.Event) -> None:
        pass

    def on_mouse_move(self, event: tk.Event) -> None:
        pass


class LineEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("LineEditor start drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = True
        self._x_start = event.x
        self._y_start = event.y
        self._x_end = event.x
```

```
        self._y_end = event.y


    def on_mouse_move(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        self._x_end = event.x
        self._y_end = event.y
        self._clear_rubber_band()
        self._owner.canvas.create_line(
            self._x_start,
            self._y_start,
            self._x_end,
            self._y_end,
            fill="black",
            dash=(4, 2),
            width=1,
            tags="rubber_band",
        )


    def on_lb_up(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        logger.debug("LineEditor end drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = False
        self._clear_rubber_band()
        shape = LineShape()
        shape.set(self._x_start, self._y_start, event.x, event.y)
        self._owner.add_shape(shape)
```

```
class RectEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("RectEditor center placed at (%d, %d).", event.x,
event.y)
        self._is_drawing = True
        self._x_start = event.x
        self._y_start = event.y
        self._x_end = event.x
        self._y_end = event.y

    def on_mouse_move(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        self._x_end = event.x
        self._y_end = event.y
        self._clear_rubber_band()
        radius_x = abs(self._x_end - self._x_start)
        radius_y = abs(self._y_end - self._y_start)
        self._owner.canvas.create_rectangle(
            self._x_start - radius_x,
            self._y_start - radius_y,
            self._x_start + radius_x,
            self._y_start + radius_y,
            fill="",
            outline="black",
            dash=(4, 2),
            width=1,
```

```
            tags="rubber_band",
        )


    def on_lb_up(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        logger.debug("RectEditor corner released at (%d, %d).",
event.x, event.y)
        self._is_drawing = False
        self._clear_rubber_band()
        radius_x = abs(event.x - self._x_start)
        radius_y = abs(event.y - self._y_start)
        shape = RectShape()
        shape.set(
            self._x_start - radius_x,
            self._y_start - radius_y,
            self._x_start + radius_x,
            self._y_start + radius_y,
        )
        self._owner.add_shape(shape)



class EllipseEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("EllipseEditor start drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = True
        self._x_start = event.x
        self._y_start = event.y
        self._x_end = event.x
```

```
        self._y_end = event.y


    def on_mouse_move(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        self._x_end = event.x
        self._y_end = event.y
        self._clear_rubber_band()
        x_min = min(self._x_start, self._x_end)
        y_min = min(self._y_start, self._y_end)
        x_max = max(self._x_start, self._x_end)
        y_max = max(self._y_start, self._y_end)
        self._owner.canvas.create_oval(
            x_min,
            y_min,
            x_max,
            y_max,
            fill="",
            outline="black",
            dash=(4, 2),
            width=1,
            tags="rubber_band",
        )


    def on_lb_up(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        logger.debug("EllipseEditor end drag at (%d, %d).", event.x,
event.y)
```

```
        self._is_drawing = False
        self._clear_rubber_band()
        shape = EllipseShape()
        shape.set(self._x_start, self._y_start, event.x, event.y)
        self._owner.add_shape(shape)
```

shape.py

```
import tkinter as tk
from abc import ABC, abstractmethod


__all__ = ["EllipseShape", "LineShape", "PointShape", "RectShape",
"Shape"]



class Shape(ABC):
    def __init__(self) -> None:
        self.xs1: int = 0
        self.ys1: int = 0
        self.xs2: int = 0
        self.ys2: int = 0

    def set(self, x1: int, y1: int, x2: int, y2: int) -> None:
        self.xs1 = x1
        self.ys1 = y1
        self.xs2 = x2
        self.ys2 = y2

    @abstractmethod
```

```
    def show(self, canvas: tk.Canvas) -> None:
        pass



class PointShape(Shape):
    def show(self, canvas: tk.Canvas) -> None:
        canvas.create_oval(
            self.xs1 - 1,
            self.ys1 - 1,
            self.xs1 + 1,
            self.ys1 + 1,
            fill="black",
            outline="black",
            tags="shape",
        )



class LineShape(Shape):
    def show(self, canvas: tk.Canvas) -> None:
        canvas.create_line(
            self.xs1,
            self.ys1,
            self.xs2,
            self.ys2,
            fill="black",
            width=1,
            tags="shape",
        )
```

```
class RectShape(Shape):
    def show(self, canvas: tk.Canvas) -> None:
        x_min = min(self.xs1, self.xs2)
        y_min = min(self.ys1, self.ys2)
        x_max = max(self.xs1, self.xs2)
        y_max = max(self.ys1, self.ys2)
        canvas.create_rectangle(
            x_min,
            y_min,
            x_max,
            y_max,
            fill="white",
            outline="black",
            width=1,
            tags="shape",
        )



class EllipseShape(Shape):
    def show(self, canvas: tk.Canvas) -> None:
        x_min = min(self.xs1, self.xs2)
        y_min = min(self.ys1, self.ys2)
        x_max = max(self.xs1, self.xs2)
        y_max = max(self.ys1, self.ys2)
        canvas.create_oval(
            x_min,
            y_min,
            x_max,
```

```
            y_max,
            fill="",
            outline="black",
            width=1,
            tags="shape",
        )
```

shape_objects_editor.py

```
import logging
import tkinter as tk
from collections.abc import Callable


from editor import (
    EllipseEditor,
    LineEditor,
    PointEditor,
    RectEditor,
    ShapeEditor,
)
from shape import Shape


__all__ = ["ShapeObjectsEditor"]


logger = logging.getLogger("Lab3.ShapeObjectsEditor")



class ShapeObjectsEditor:
    MAX_CAPACITY = 115
```

```
    def __init__(
        self,
        canvas: tk.Canvas,
        on_mode_change: Callable[[str, str], None] | None = None,
    ) -> None:
        self.canvas = canvas
        self._on_mode_change = on_mode_change
        self._pcshape: list[Shape] = []

        self._editors: dict[str, ShapeEditor] = {
            "point": PointEditor(self),
            "line": LineEditor(self),
            "rect": RectEditor(self),
            "ellipse": EllipseEditor(self),
        }
        self._shape_labels: dict[str, str] = {
            "point": "Крапка",
            "line": "Лінія",
            "rect": "Прямокутник",
            "ellipse": "Еліпс",
        }

        self._current_type_name: str = "point"
        self._current_editor: ShapeEditor =
self._editors[self._current_type_name]

    @property
    def current_type_name(self) -> str:
```

```
        return self._current_type_name


    @property
    def current_label(self) -> str:
        return self._shape_labels.get(self._current_type_name, "")


    def start_editor(self, shape_type: str) -> None:
        if shape_type in self._editors:
            logger.info("Switching mode to %s.", shape_type)
            self._current_editor = self._editors[shape_type]
            self._current_type_name = shape_type
            if self._on_mode_change is not None:
                self._on_mode_change(shape_type, self.current_label)
        else:
            logger.error("Unknown editor type requested: '%s'.",
shape_type)


    def on_lb_down(self, event: tk.Event) -> None:
        self._current_editor.on_lb_down(event)


    def on_lb_up(self, event: tk.Event) -> None:
        self._current_editor.on_lb_up(event)


    def on_mouse_move(self, event: tk.Event) -> None:
        self._current_editor.on_mouse_move(event)


    def on_paint(self) -> None:
        self.canvas.delete("shape")
        for shape in self._pcshape:
```

```
            shape.show(self.canvas)


    def add_shape(self, shape: Shape) -> None:
        if len(self._pcshape) < self.MAX_CAPACITY:
            self._pcshape.append(shape)
            logger.info(
                "Shape added to dynamic array. Current size: %d/%d.",
                len(self._pcshape),
                self.MAX_CAPACITY,
            )
            self.on_paint()
        else:
            logger.warning("Dynamic array capacity reached (%d).",
self.MAX_CAPACITY)


    def on_init_menu_popup(self, menu_var: tk.StringVar) -> None:
        menu_var.set(self._current_type_name)
        logger.debug("Menu state synced: current mode '%s'.",
self._current_type_name)
```

toolbar.py

```
import logging
import math
import tkinter as tk
from collections.abc import Callable
from tkinter import ttk


__all__ = ["ToolTip", "Toolbar"]
```

```
logger = logging.getLogger("Lab3.Toolbar")



class ToolTip:
    def __init__(
        self,
        widget: tk.Widget,
        text: str,
        notify_callback: Callable[[str], None] | None = None,
    ) -> None:
        self._widget = widget
        self._text = text
        self._notify_callback = notify_callback
        self._tip_window: tk.Toplevel | None = None

        self._widget.bind("<Enter>", self.on_notify_enter)
        self._widget.bind("<Leave>", self.on_notify_leave)

    def on_notify_enter(self, _event: tk.Event) -> None:
        if self._notify_callback is not None:
            self._notify_callback(self._text)
        if self._tip_window is not None:
            return
        x = self._widget.winfo_rootx() + 10
        y = self._widget.winfo_rooty() + self._widget.winfo_height() +
4
        self._tip_window = tk.Toplevel(self._widget)
        self._tip_window.wm_overrideredirect(True)
        self._tip_window.wm_geometry(f"+{x}+{y}")
```

```
        label = tk.Label(
            self._tip_window,
            text=self._text,
            background="#FFFFE0",
            foreground="#000000",
            relief=tk.SOLID,
            borderwidth=1,
            font=("Segoe UI", 9),
            padx=6,
            pady=2,
        )
        label.pack()

    def on_notify_leave(self, _event: tk.Event) -> None:
        if self._tip_window is not None:
            self._tip_window.destroy()
            self._tip_window = None


class Toolbar(ttk.Frame):
    def __init__(
        self,
        parent: tk.Widget,
        on_select_tool: Callable[[str], None],
    ) -> None:
        super().__init__(parent, relief=tk.RAISED, borderwidth=1)
        self._on_select_tool = on_select_tool
        self._icons: dict[str, tk.PhotoImage] = {}
        self._buttons: dict[str, tk.Button] = {}
```

```
        self._tooltips: list[ToolTip] = []


        self._build_icons()
        self._build_buttons()


    def _build_icons(self) -> None:
        size = 24


        point_img = tk.PhotoImage(width=size, height=size)
        for y in range(size):
            for x in range(size):
                if (x - 12) ** 2 + (y - 12) ** 2 <= 9:
                    point_img.put("#000000", (x, y))
        self._icons["point"] = point_img


        line_img = tk.PhotoImage(width=size, height=size)
        for i in range(4, 20):
            line_img.put("#000000", (i, 23 - i))
            line_img.put("#000000", (i, 22 - i))
        self._icons["line"] = line_img


        rect_img = tk.PhotoImage(width=size, height=size)
        for y in range(5, 19):
            for x in range(4, 20):
                if x in (4, 19) or y in (5, 18):
                    rect_img.put("#000000", (x, y))
                else:
                    rect_img.put("#FFFFFF", (x, y))
        self._icons["rect"] = rect_img
```

```
        ellipse_img = tk.PhotoImage(width=size, height=size)
        rx, ry = 8.0, 5.5
        cx, cy = 12.0, 12.0
        for angle in range(360):
            rad = math.radians(angle)
            x = round(cx + rx * math.cos(rad))
            y = round(cy + ry * math.sin(rad))
            ellipse_img.put("#000000", (x, y))
        self._icons["ellipse"] = ellipse_img


    def _build_buttons(self) -> None:
        tools = [
            ("point", "Крапка"),
            ("line", "Лінія"),
            ("rect", "Прямокутник"),
            ("ellipse", "Еліпс"),
        ]
        for shape_type, tooltip_text in tools:
            btn = tk.Button(
                self,
                image=self._icons[shape_type],
                width=28,
                height=28,
                relief=tk.RAISED,
                bd=1,
                command=lambda t=shape_type:
self._on_button_pressed(t),
            )
```

```
            btn.pack(side=tk.LEFT, padx=2, pady=2)
            self._buttons[shape_type] = btn
            self._tooltips.append(ToolTip(btn, tooltip_text,
self.on_notify))

    def _on_button_pressed(self, shape_type: str) -> None:
        logger.info("Toolbar button clicked: '%s'.", shape_type)
        self._on_select_tool(shape_type)

    def on_notify(self, tooltip_text: str) -> None:
        logger.debug("OnNotify triggered: displaying tooltip '%s'.",
tooltip_text)

    def set_active_button(self, shape_type: str) -> None:
        for key, btn in self._buttons.items():
            if key == shape_type:
                btn.config(relief=tk.SUNKEN, bg="#D0E0F0")
            else:
                btn.config(relief=tk.RAISED, bg="SystemButtonFace")
```

main.py

```
import logging
import tkinter as tk
from tkinter import messagebox

from shape_objects_editor import ShapeObjectsEditor
from toolbar import Toolbar

logging.basicConfig(
```

```
    level=logging.DEBUG,
    format="%(asctime)s [%(levelname)s] (%(name)s): %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger("Lab3.Main")



class MainWindow(tk.Tk):
    BASE_TITLE = "OOP_lab3"


    def __init__(self) -> None:
        super().__init__()
        logger.info("Initializing MainWindow.")
        self.title(self.BASE_TITLE)
        self.geometry("720x540")


        self._menu_shape_var = tk.StringVar(value="point")


        self._init_toolbar()
        self._init_canvas()
        self._editor = ShapeObjectsEditor(
            self._canvas,
            on_mode_change=self._on_editor_mode_changed,
        )
        self._init_menu()
        self._bind_events()


        self._editor.start_editor("point")
```

```
    def _init_toolbar(self) -> None:
        logger.debug("Configuring Toolbar.")
        self._toolbar = Toolbar(
            self,
            on_select_tool=lambda shape_type:
self._editor.start_editor(shape_type),
        )
        self._toolbar.pack(side=tk.TOP, fill=tk.X)


    def _init_canvas(self) -> None:
        self._canvas = tk.Canvas(self, bg="#FFFFFF",
cursor="crosshair")
        self._canvas.pack(side=tk.TOP, fill=tk.BOTH, expand=True)


    def _init_menu(self) -> None:
        logger.debug("Configuring menu bar.")
        menubar = tk.Menu(self)


        file_menu = tk.Menu(menubar, tearoff=0)
        file_menu.add_command(label="Вихід", command=self.destroy)
        menubar.add_cascade(label="Файл", menu=file_menu)


        objects_menu = tk.Menu(
            menubar,
            tearoff=0,
            postcommand=lambda:
self._editor.on_init_menu_popup(self._menu_shape_var),
        )
        objects_menu.add_radiobutton(
            label="Крапка",
```

```
            variable=self._menu_shape_var,
            value="point",
            command=lambda: self._editor.start_editor("point"),
        )
        objects_menu.add_radiobutton(
            label="Лінія",
            variable=self._menu_shape_var,
            value="line",
            command=lambda: self._editor.start_editor("line"),
        )
        objects_menu.add_radiobutton(
            label="Прямокутник",
            variable=self._menu_shape_var,
            value="rect",
            command=lambda: self._editor.start_editor("rect"),
        )
        objects_menu.add_radiobutton(
            label="Еліпс",
            variable=self._menu_shape_var,
            value="ellipse",
            command=lambda: self._editor.start_editor("ellipse"),
        )
        menubar.add_cascade(label="Об'єкти", menu=objects_menu)

        help_menu = tk.Menu(menubar, tearoff=0)
        help_menu.add_command(label="Про програму",
command=self._on_about)
        menubar.add_cascade(label="Довідка", menu=help_menu)
```

```
        self.config(menu=menubar)


    def _on_editor_mode_changed(self, shape_type: str, label_ukr: str)
-> None:
        self.title(f"{self.BASE_TITLE} - Об'єкт: {label_ukr}")
        self._menu_shape_var.set(shape_type)
        self._toolbar.set_active_button(shape_type)


    def _bind_events(self) -> None:
        self._canvas.bind("<Button-1>", self._editor.on_lb_down)
        self._canvas.bind("<B1-Motion>", self._editor.on_mouse_move)
        self._canvas.bind("<ButtonRelease-1>", self._editor.on_lb_up)
        self._canvas.bind("<Configure>", lambda _event:
self._editor.on_paint())

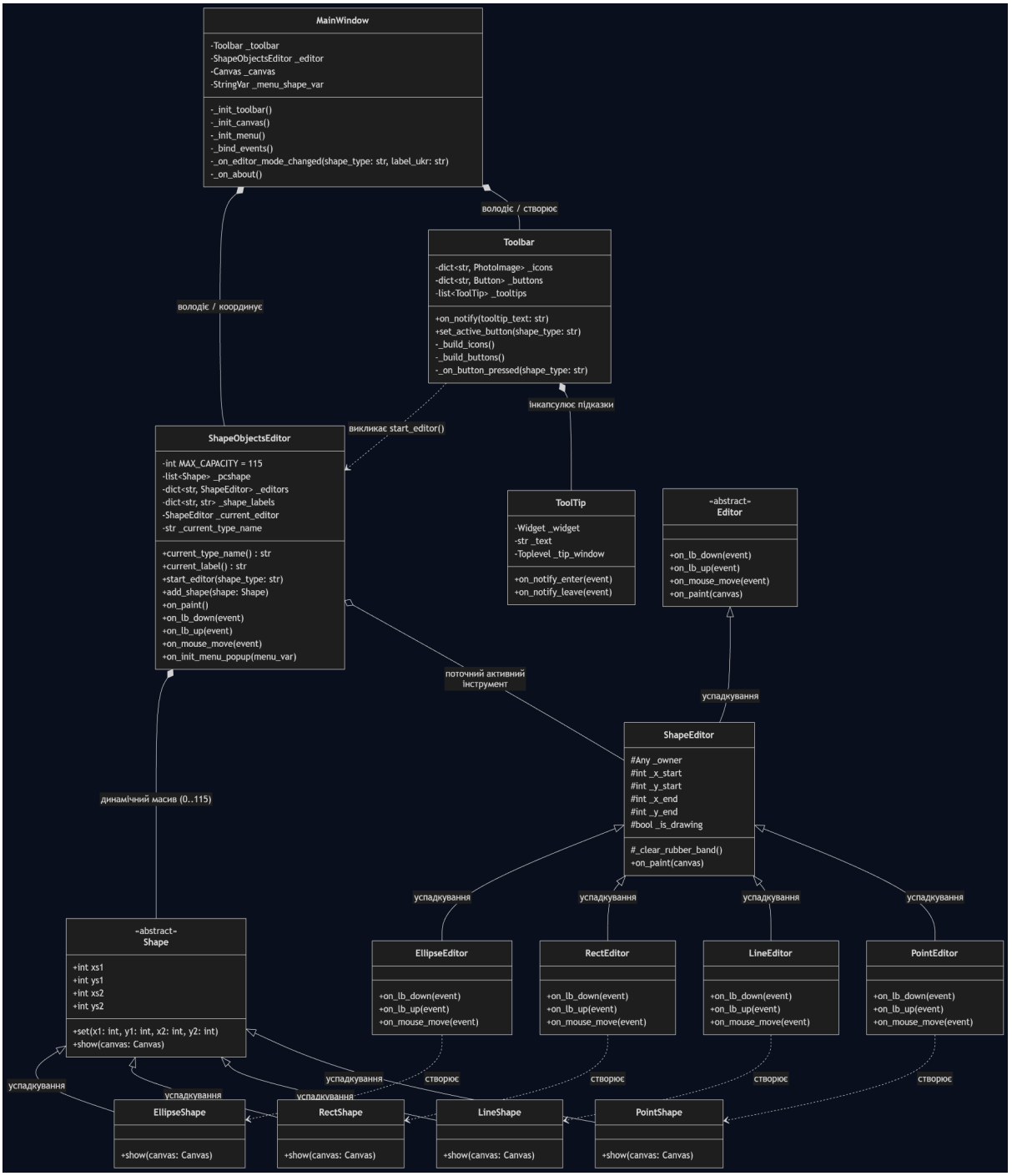

    def _on_about(self) -> None:
        logger.info("Displaying About dialog.")
        messagebox.showinfo(
            "Про програму",
            "Лабораторна робота №3\nРозробка інтерфейсу
користувача\nВаріант 14 (Ж = 15, N = 115)",
        )



def main() -> None:
    logger.info("Starting application loop.")
    app = MainWindow()
    app.mainloop()
    logger.info("Application loop ended.")
```

```
if __name__ == "__main__":
    main()
```

Діаграма класів та залежностей



Опис діаграми та взаємодії класів:

1. **Інкапсуляція подій користувача:** Усі процедури обробки повідомлень реалізовані як методи відповідних класів. Клас ToolTip відловлює наведення курсора (<Enter>, <Leave>) і викликає метод-член класу Toolbar.on_notify(), забезпечуючи вимогу п. 2 щодо обробника OnNotify.

2. **Панель інструментів Toolbar:** Інкапсулює створення піктограм для кожної фігури, керує активним візуальним станом кнопок (set_active_button) та передає сигнал вибору інструмента через зворотний виклик on_select_tool.
3. **Диспетчеризація інструментів:** Контролер ShapeObjectsEditor використовує таблицю диспетчеризації self._editors та єдиний метод start_editor(shape_type), синхронізуючи заголовок вікна, активну кнопку в Toolbar та позначку у випадаючому меню.
4. **Динамічний масив об'єктів:** Поле _pcshape реалізоване як динамічний список list[Shape], який збільшується методом append() до максимальної межі $N = 115$. Метод on_paint() поліморфно відмальовує всі об'єкти без аналізу їхніх типів.

скріншоти

Головне вікно програми з панеллю інструментів Toolbar та відображенням режиму в заголовку



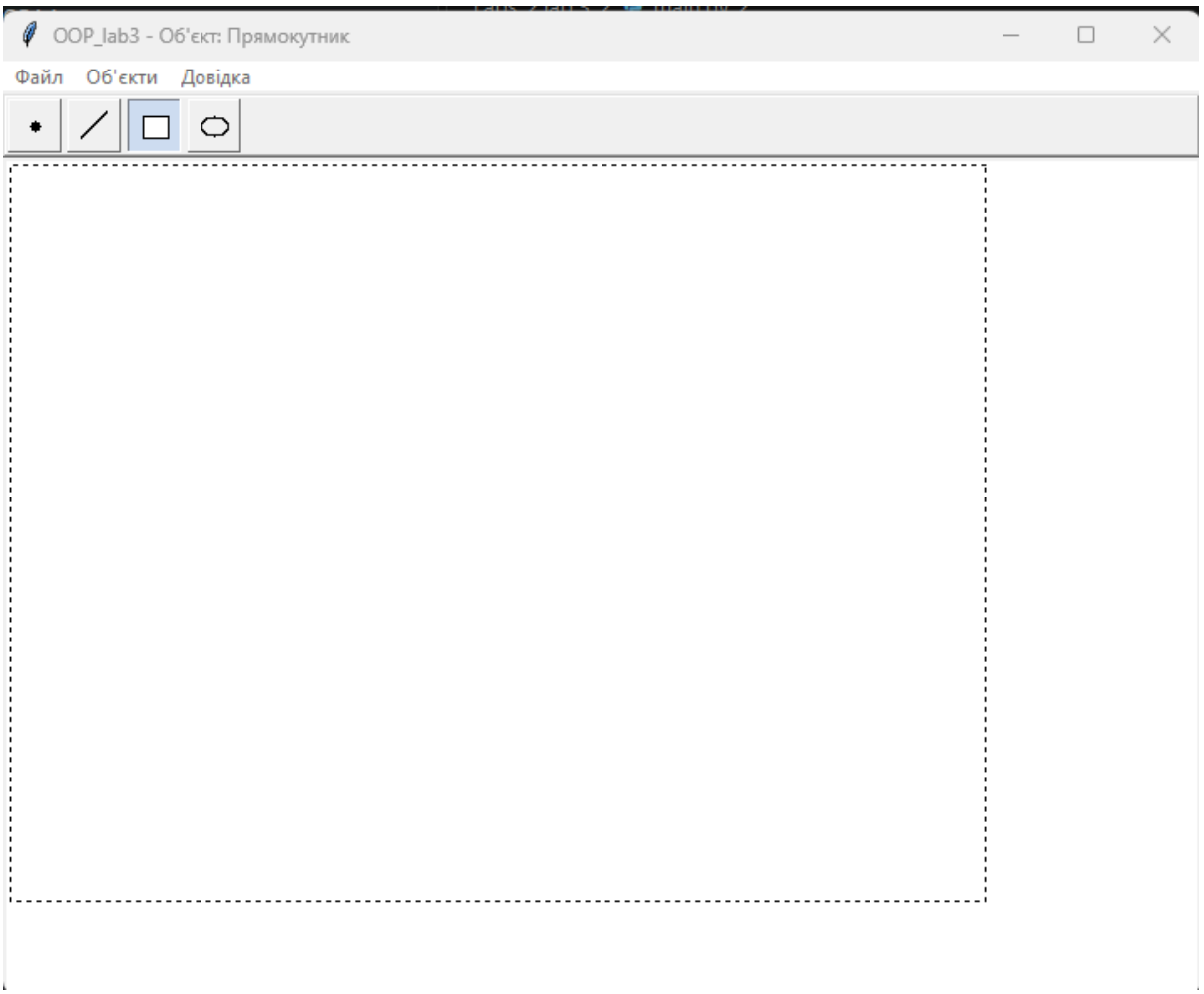
Відображення спливаючої підказки (ToolTip) при наведенні на кнопку інструмента


OOP_lab3 - Об'єкт: Крапка
Файл
Об'єкти
Довідка
Прямокутник

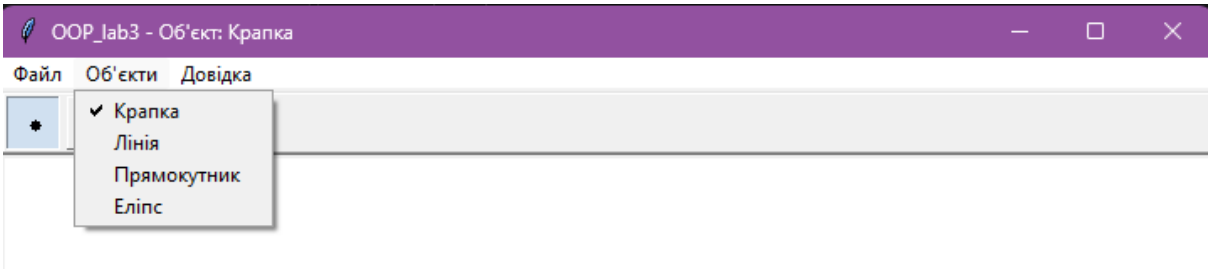
Процес побудови об'єкта з чорним пунктирним «гумовим» слідом



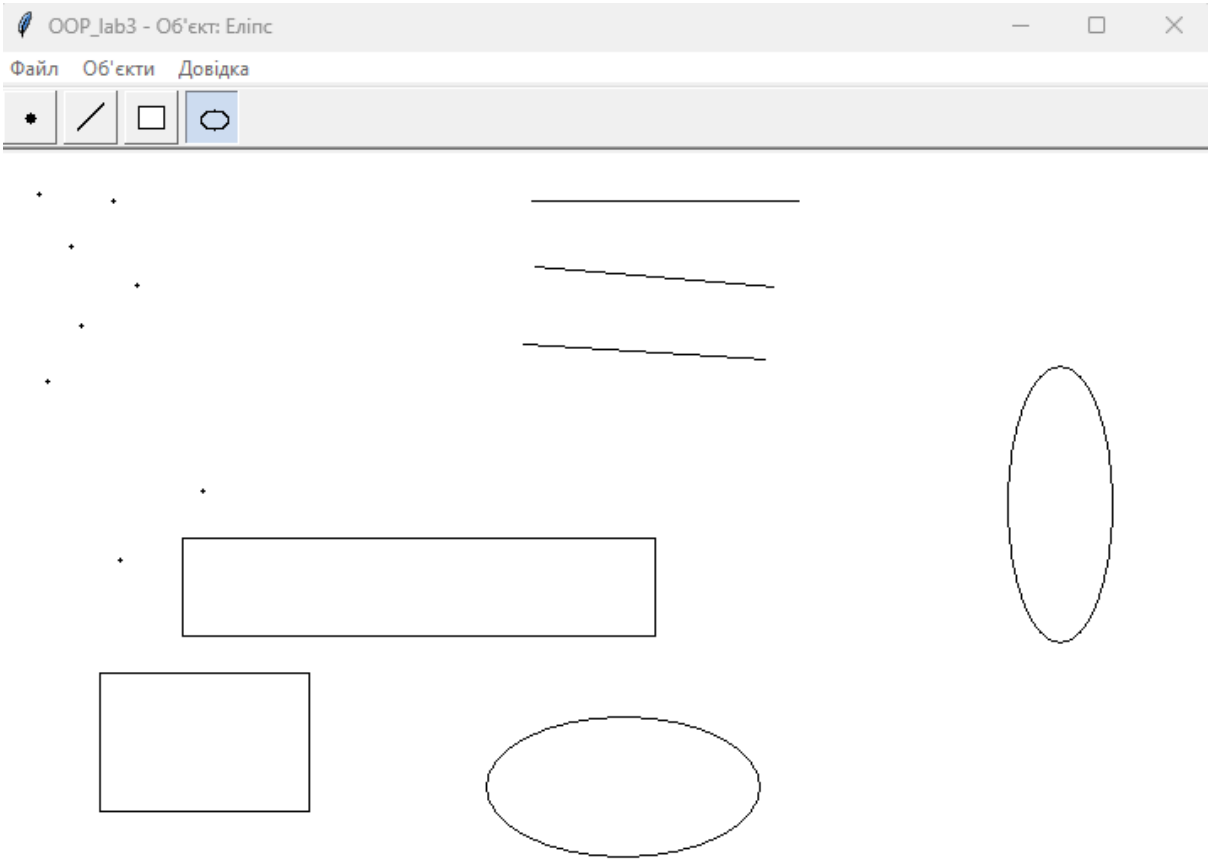
Демонстрація введення прямокутника від центру до кута



Синхронізація елементів інтерфейсу (Toolbar, Меню «Об'єкти», Заголовок)



Результуюче полотно з побудованими фігурами всіх чотирьох типів



Висновки

Під час виконання лабораторної роботи №3 було розширено функціональність об'єктно-орієнтованого векторного графічного редактора шляхом проектування та інтеграції компонентів графічного інтерфейсу користувача (GUI).